《Ai Agent》第3-18节：动态执行智能体任务

来自： 码农会锁

小傅哥 2025年09月20日 14:00

本章重点 ：★★★☆☆
课程视频 ：https://t.zsxq.com/j116r
代码分支 ：https://gitcode.net/KnowledgePlanet/ai-agent-station-study/-/tree/3-18-agent-task-job
工程代码 ：https://gitcode.net/KnowledgePlanet/ai-agent-station-study

版权说明 ：©本项目与星球签约合作，受《中华人民共和国著作权法实施条例》版权法保护，禁止任何理由和任何方式公开(public)源码、资料、视频等小傅哥发布的星球内容到Github、Gitee等各类平台，违反可追究进一步的法律责任。

作者：小傅哥
博客：https://bugstack.cn

> 沉淀、分享、成长，让自己和他人都能有所收获！😄

一、本章诉求
二、功能流程
三、编码实现
1. 工程结构
2. 修改说明
3. 库表数据
4. 核心实现
四、测试验证
1. 配置说明
2. 启动测试
五、读者作业

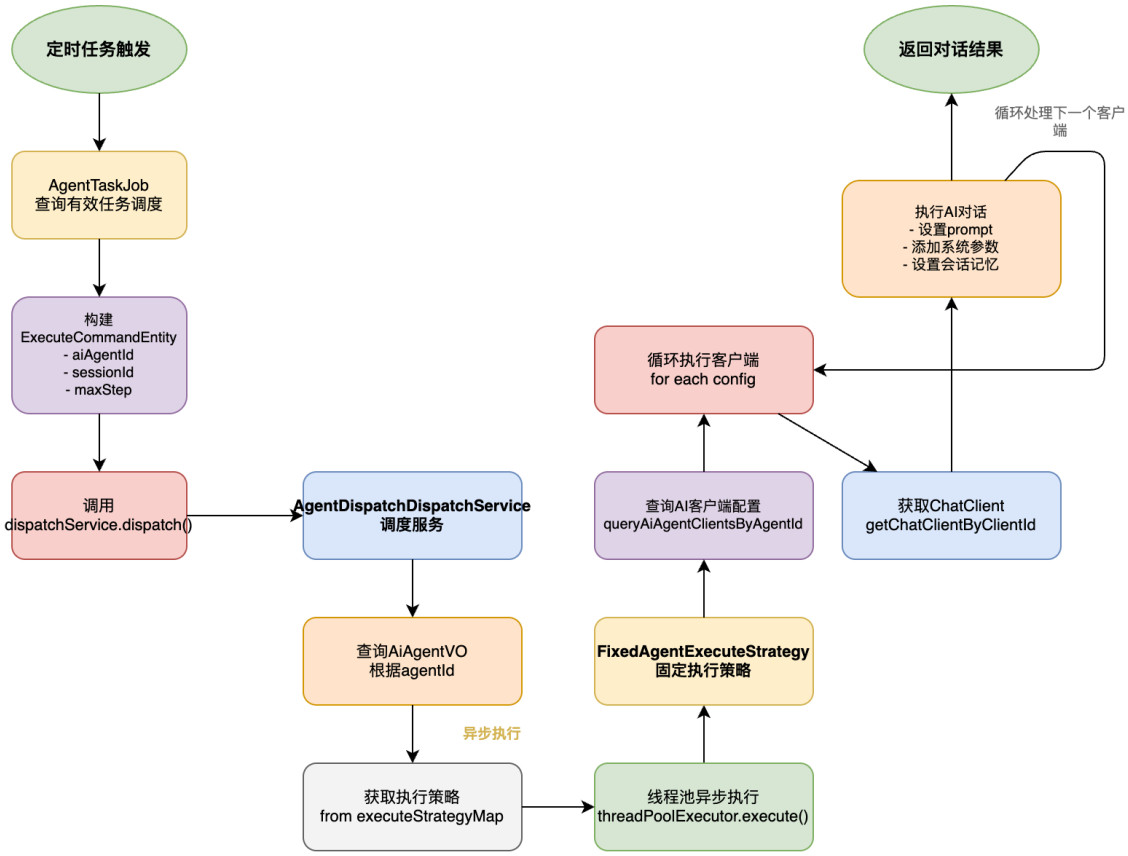
一、本章诉求

本节首先新增一个基于数据库固定配置步骤的简单循环执行智能体策略。随后，结合数据库中配置的智能体定时执行参数，以及工程中引入的扳手工程动态任务组件，实现对智能体动态任务的调度与执行。

那么，这里有一个前置学习，《通用技术组件 - 🔧 扳手工程》第5节：任务调度组件 https://t.zsxq.com/bLkoF - 需要先完成下，之后就可以在智能体项目中使用了。

二、功能流程

如图，Ai Agent 动态任务执行过程；



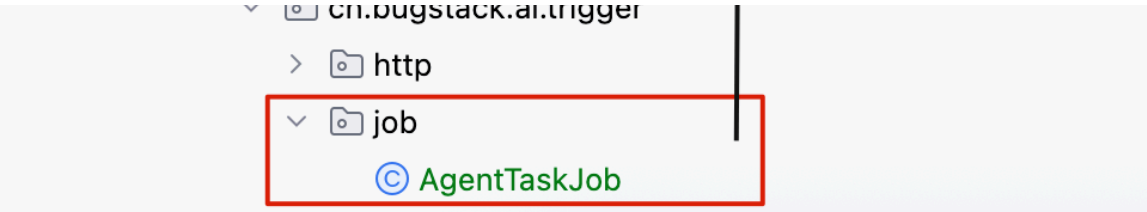
首先，我们在项目中引入了扳手工程的任务调度组件。在调度组件内，配置检索出数据库表内可执行的任务以及对应的参数，之后由定时任务管理。

之后，本节我们又增加了一个新的简单的 Agent 执行策略，依次循环数据库 Agent 配置的客户端。这个场景很适合工作流编排，把固定要执行的客户端，依次写入到数据库配置，就可以按照固定步骤执行了。这也是咱们在早期分析 Agent 执行方式的一种。

三、编码实现

1. 工程结构

ai-agent-station-study
3-18-agent-task-job
Project
ai-agent-station-study-domain
src
main
java
cn.bugstack.ai.domain
agent
adapter
model
service
armory
dispatch
AgentDispatchDispatchService
execute
auto
fixed
FixedAgentExecuteStrategy
固定步骤 Agent
flow
step
FlowAgentExecuteStrategy
task
AiAgentTaskService
IAgentDispatchService
IExecuteStrategy
ITaskService
package-info.java
调度执行器
yyy
target
pom.xml
ai-agent-station-study-infrastructure
ai-agent-station-study-trigger
src
main
java



本节在 trigger 触发器层，在 job 模块添加了一个 AgentTaskJob 实现类。这个任务会实现扳手工程**任务调度组件** 的接口，之后这个类就会被任务调度管理了。

此外本节除了实现了 job 任务，还增加了一个新的 FixedAgentExecuteStrategy 执行 Agent 模型。这个模式会按照数据库里配置的步骤执行。

2. 修改说明

1. 引入扳手工程的任务调度组件，这里有2个方式引入，一个是通过 pom 配置文件，引入 `xfg-wrench-starter-task-job` 另外一个是把扳手工程任务调度组件的代码复制到 types 模块下使用。
2. 在 application-dev.yml 文件中，增加小傅哥扳手工程 task.job 任务配置，这部分可以参考课程代码配置。
3. 增加一个 FixedAgentExecuteStrategy 策略实现类，循环方式执行数据库配置的客户端。
4. domain/agent/task 增加一个实现类，查询数据库表 ai_agent_task_schedule 配置的任务执行数据。通过执行数据的查询，就可以把数据写入到任务组件里执行。
5. 在 trigger 层，job 下编写一个实现了 ITaskDataProvider 的 AgentTaskJob 实现类。

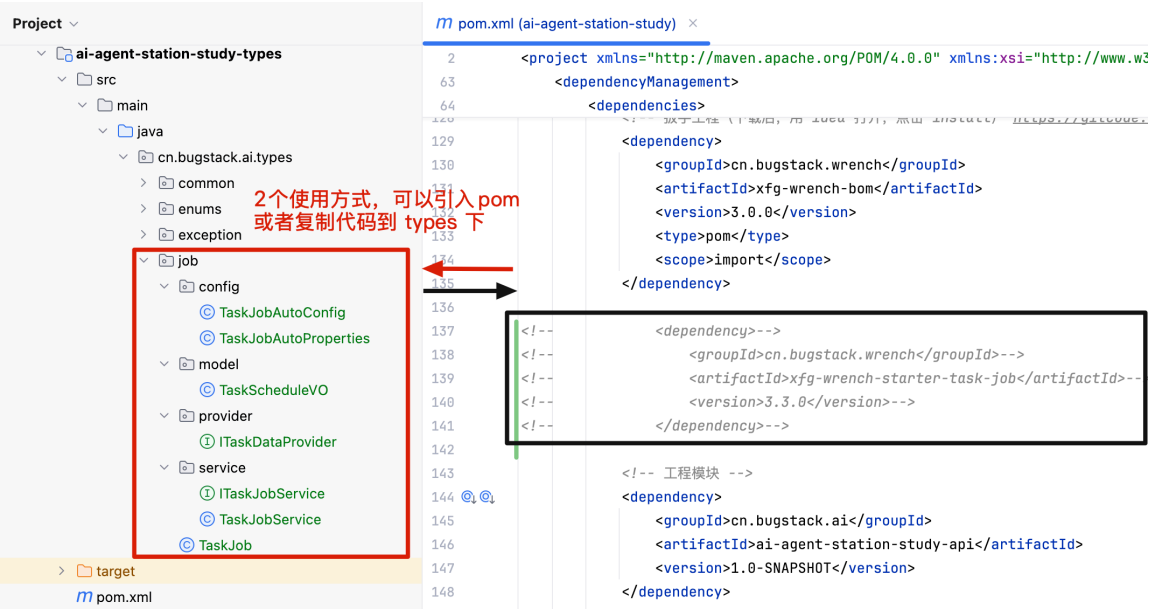
3. 库表数据

ai_agent 以及各项配置表，增加智能体 ID = 6 的配置。这是一个自动发贴的 Agent

之后还需要一个 schedule 表配置执行任务的，尤其是 cron 表达式，明确执行任务的时间频次。

4. 核心实现

4.1 引入组件



需要引入扳手工程的任务调度组件到 ai-agent-station-study 中使用。

4.2 固定模式（Agent）

```java
@Service("fixedAgentExecuteStrategy")
public class FixedAgentExecuteStrategy implements IExecuteStrategy {

    @Resource
    private IAgentRepository repository;

    @Resource
    protected ApplicationContext applicationContext;

    public static final String CHAT_MEMORY_CONVERSATION_ID_KEY = "chat_memory_conversation_id";
    public static final String CHAT_MEMORY_RETRIEVE_SIZE_KEY = "chat_memory_response_size";

    @Override
    public void execute(ExecuteCommandEntity requestParameter, ResponseBodyEmitter emitter) throws
        // 1. 获取配置客户端
        List<AiAgentClientFlowConfigVO> aiAgentClientList = repository.queryAiAgentClientsByAgentI

        // 2. 循环执行客户端
        String content = "";

        for (AiAgentClientFlowConfigVO config : aiAgentClientList) {
            ChatClient chatClient = getChatClientByClientId(config.getClientId());

            content = chatClient.prompt(requestParameter.getMessage() + ", " + content)
                    .system(s -> s.param("current_date", LocalDate.now().toString()))
                    .advisors(a -> a
                            .param(CHAT_MEMORY_CONVERSATION_ID_KEY, requestParameter.getSessionId(
                            .param(CHAT_MEMORY_RETRIEVE_SIZE_KEY, 100))
                    .call().content();

            log.info("智能体对话进行，客户端ID {}", requestParameter.getAiAgentId());
        }

        log.info("智能体对话请求，结果 {} {}", requestParameter.getAiAgentId(), content);
    }

    private ChatClient getChatClientByClientId(String clientId) {
        return getBean(AiAgentEnumVO.AI_CLIENT.getBeanName(clientId));
    }

    private <T> T getBean(String beanName) {
        return (T) applicationContext.getBean(beanName);
    }

}
```

这个模式就是最开始第三阶段 ai-agent-station 简单的 agent 执行模式，虽然这种模式很固定，但可以很准确的执行用户编排动作，意图明确比较节省调用ai消耗的 token 量。

注意，数据库表对于此类 agent 也要配置对应的策略。ai_agent -> fixedAgentExecuteStrategy

4.3 任务调度

```java
@Service
public class AgentTaskJob implements ITaskDataProvider {

    @Resource
    private ITaskService taskService;

    @Resource
    private IAgentDispatchService dispatchService;
```

```java
    @Override
    public List<TaskScheduleVO> queryAllValidTaskSchedule() {
        List<AiAgentTaskScheduleVO> aiAgentTaskScheduleVOS = taskService.queryAllValidTaskSchedule
        List<TaskScheduleVO> result = new ArrayList<>();
        for (AiAgentTaskScheduleVO aiAgentTaskScheduleVO : aiAgentTaskScheduleVOS) {
            TaskScheduleVO taskScheduleVO = new TaskScheduleVO();
            taskScheduleVO.setId(aiAgentTaskScheduleVO.getId());
            taskScheduleVO.setDescription(aiAgentTaskScheduleVO.getDescription());
            taskScheduleVO.setCronExpression(aiAgentTaskScheduleVO.getCronExpression());
            taskScheduleVO.setTaskParam(aiAgentTaskScheduleVO.getTaskParam());
            taskScheduleVO.setTaskLogic(() -> {
                try {
                    dispatchService.dispatch(
                            ExecuteCommandEntity.builder()
                                    .aiAgentId(aiAgentTaskScheduleVO.getAgentId())
                                    .sessionId(String.valueOf(System.nanoTime()))
                                    .maxStep(1)
                                    .build(), new ResponseBodyEmitter());
                } catch (Exception e) {
                    log.error("任务执行失败", e);
                }

            });

            result.add(taskScheduleVO);
        }
        return result;
    }

    @Override
    public List<Long> queryAllInvalidTaskScheduleIds() {
        return taskService.queryAllInvalidTaskScheduleIds();
    }

}
```

实现任务调度组件 ITaskDataProvider 接口，AgentTaskJob 类就可以加载任务并自动根据 cron 表达式频次进行执行。

四、测试验证

1. 配置说明

```yaml
ai:
  openai:
    base-url: https://apis.itedus.cn
    api-key: sk-Pd9r8vPQVL8eWyJtF2E79277Ef874b8cAc2d2a038186Dc7b
  agent:
    auto-config:
      enabled: true
      client-ids: 6101
```

配置客户端ID，6101 另外注意这个 Agent 需要 CSDN MCP 和 微信公众号 MCP 服务。

2. 启动测试

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
```

```
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.4.3)

25-09-16.06:11:50.066 [main            ] INFO  Application            - Starting Application using
25-09-16.06:11:51.204 [main            ] INFO  PgVectorStore          - Initializing PGVectorStore
25-09-16.06:11:51.205 [main            ] INFO  PgVectorStore          - vectorTableValidationsEnab
25-09-16.06:11:51.269 [main            ] INFO  TaskJobAutoConfig      - xfg-wrench, 任务调度器初始化完
25-09-16.06:11:51.271 [main            ] INFO  TaskJobService         - 开始初始化任务调度配置
25-09-16.06:11:51.279 [main            ] INFO  HikariDataSource       - MainHikariPool - Starting.
25-09-16.06:11:51.379 [main            ] INFO  HikariPool             - MainHikariPool - Added con
25-09-16.06:11:51.379 [main            ] INFO  HikariDataSource       - MainHikariPool - Start com
25-09-16.06:11:51.397 [main            ] INFO  TaskJobService         - 开始调度任务, ID: 1, 描述: 自
25-09-16.06:11:51.400 [main            ] INFO  TaskJobService         - 任务调度成功 (函数式), ID: 1
25-09-16.06:11:51.400 [main            ] INFO  TaskJobService         - 任务调度配置初始化完成, 已加载任
25-09-16.06:11:51.400 [main            ] INFO  TaskJobAutoConfig      - xfg-wrench, 任务调度作业初始化
25-09-16.06:11:51.575 [main            ] INFO  TomcatWebServer        - Tomcat started on port 809
25-09-16.06:11:51.580 [main            ] INFO  Application            - Started Application in 1.7
25-09-16.06:11:51.581 [test-task-scheduler-2] INFO  TaskJobService         - 开始刷新任务调度配置 (动态
25-09-16.06:11:51.582 [main            ] INFO  AiAgentAutoConfiguration - AI Agent 自动装配开始, 配置
25-09-16.06:11:51.583 [main            ] INFO  AiAgentAutoConfiguration - 开始自动装配AI客户端, 客户端
25-09-16.06:11:51.583 [test-task-scheduler-2] INFO  TaskJobService         - 任务调度配置刷新完成, 当前
25-09-16.06:11:51.584 [pool-2-thread-1 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client_api
25-09-16.06:11:51.584 [pool-2-thread-2 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client_mod
25-09-16.06:11:51.584 [pool-2-thread-3 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client_too
25-09-16.06:11:51.584 [pool-2-thread-4 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client_sys
25-09-16.06:11:51.584 [pool-2-thread-5 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client_adv
25-09-16.06:11:51.585 [pool-2-thread-6 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client) [6
25-09-16.06:11:51.657 [main            ] INFO  RootNode               - Ai Agent 构建, 数据加载节点{"
25-09-16.06:11:51.658 [main            ] INFO  AiClientApiNode        - Ai Agent 构建节点, API 接口请
25-09-16.06:11:51.659 [main            ] INFO  AbstractArmorySupport  - 成功注册Bean: ai_client_api_
25-09-16.06:11:51.659 [main            ] INFO  AiClientToolMcpNode    - Ai Agent 构建节点, Tool MCP
25-09-16.06:11:51.731 [HttpClient-10-Worker-0] INFO  McpAsyncClient         - Server response with
25-09-16.06:11:51.739 [main            ] INFO  AiClientToolMcpNode    - Tool SSE MCP Initialized I
25-09-16.06:11:51.740 [main            ] INFO  AbstractArmorySupport  - 成功注册Bean: ai_client_too
25-09-16.06:11:51.758 [HttpClient-11-Worker-2] INFO  McpAsyncClient         - Server response with
25-09-16.06:11:51.761 [main            ] INFO  AiClientToolMcpNode    - Tool SSE MCP Initialized I
25-09-16.06:11:51.761 [main            ] INFO  AbstractArmorySupport  - 成功注册Bean: ai_client_too
25-09-16.06:11:51.761 [main            ] INFO  AiClientModelNode      - Ai Agent 构建节点, Mode 对话
25-09-16.06:11:51.781 [main            ] INFO  AbstractArmorySupport  - 成功注册Bean: ai_client_mod
25-09-16.06:11:51.782 [main            ] INFO  AiClientAdvisorNode    - Ai Agent 构建节点, Advisor 
25-09-16.06:11:51.785 [main            ] INFO  AbstractArmorySupport  - 成功注册Bean: ai_client_adv
25-09-16.06:11:51.803 [main            ] INFO  AbstractArmorySupport  - 成功注册Bean: ai_client_adv
25-09-16.06:11:51.803 [main            ] INFO  AiClientNode           - Ai Agent 构建节点, 客户端{"cc
25-09-16.06:11:51.806 [main            ] INFO  AbstractArmorySupport  - 成功注册Bean: ai_client_610
25-09-16.06:11:51.806 [main            ] INFO  AiAgentAutoConfiguration - AI Agent 自动装配完成
25-09-16.06:11:55.005 [test-task-scheduler-1] INFO  TaskJobService         - 开始执行任务 (函数式), ID
25-09-16.06:12:10.852 [test-task-scheduler-1] INFO  TaskJobService         - 任务执行完成 (函数式), ID
25-09-16.06:12:13.141 [pool-2-thread-7 ] INFO  HikariDataSource       - PgVectorHikariPool - Start
25-09-16.06:12:13.717 [pool-2-thread-7 ] INFO  HikariPool             - PgVectorHikariPool - Added
25-09-16.06:12:13.717 [pool-2-thread-7 ] INFO  HikariDataSource       - PgVectorHikariPool - Start
25-09-16.06:12:15.001 [test-task-scheduler-2] INFO  TaskJobService         - 开始执行任务 (函数式), ID
```

以上是整个系统启动的日志，构建和执行。之后会有定时任务执行完成发帖动作。

五、读者作业

简单作业：完成本节内容，引入任务组件执行配置的自动化执行 Agent 任务。

复杂作业：尝试添加一个新的 Ai Agent 玩法，之后通过任务定时执行。你可以通过 mcp 网站，获得非常多的服务能力进行配置使用。

知识星球